S Y S T E M A R C H I T E C T U R E & T E C H N I C A L D O C U M E N T A T I O N

RealtimeChatSystem

Backend Engineering Reference · v1.0

ASP.NET Core 8 · SignalR · Redis · RabbitMQ

SQL Server · EF Core · JWT Authentication · Docker

Clean Architecture · Scalable Real-Time Backend

Audience: Backend Engineers · Architecture Review · Technical Onboarding

Table of Contents

1. Executive Summary

RealtimeChatSystem is a distributed, production-grade backend for real-time messaging, engineered with Clean Architecture principles and built to operate correctly under concurrent load across multiple server instances.

The system implements the core primitives of a messaging platform — 1-to-1 and group conversations, real-time message delivery, online presence tracking, typing indicators, and a durable message lifecycle — using a technology stack chosen deliberately for the guarantees each component provides.

Technical Goals

Goal	Engineering Response
Real-time delivery	SignalR WebSockets with Redis backplane for multi-instance broadcast
Message durability	SQL Server as source of truth; RabbitMQ for at-least-once delivery
Horizontal scalability	Stateless JWT auth; Redis backplane; queue-based consumer decoupling
Offline resilience	Reconnect delivery sweep; RabbitMQ durable queues for push notification pipeline
Architectural integrity	Clean Architecture; dependency inversion; persistence-ignorant domain model
Operational safety	Structured exception handling; sanitised error responses; BCrypt password hashing

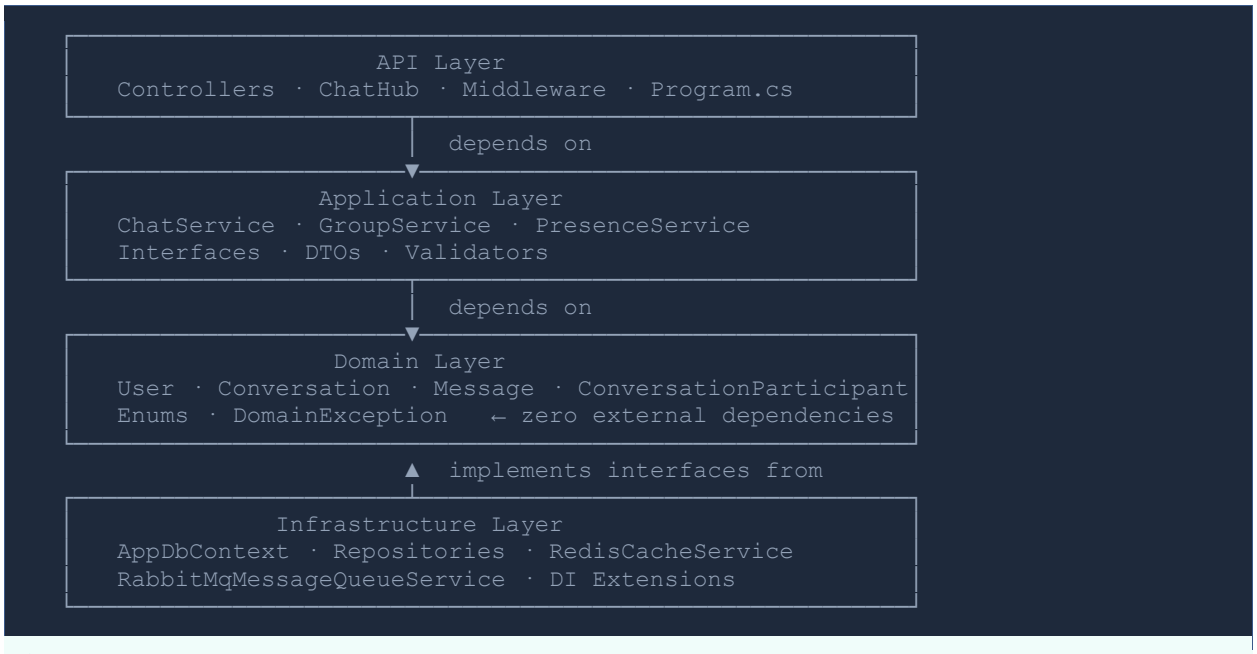
2. System Architecture Overview

The system is structured as four concentric layers. The dependency rule is absolute: outer layers depend on inner layers; inner layers have no knowledge of outer layers. This inversion is enforced structurally — the Domain layer has zero NuGet package references; the Application layer references only the Domain.

Layer Responsibilities

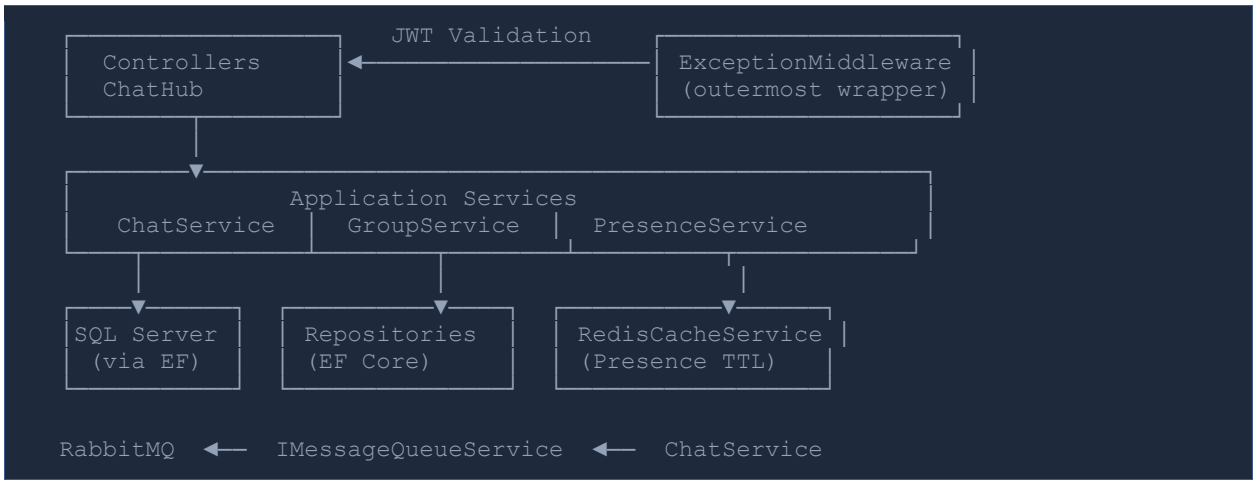
Layer	Primary Concern	Contents
Domain	Business rules & invariants	Entities, Enums, DomainException — zero external dependencies
Application	Use-case orchestration	Services, Interfaces, DTOs, Validators — depends only on Domain
Infrastructure	External integrations	EF Core, Redis, RabbitMQ, Repositories — implements Application interfaces
API	Transport & entry point	Controllers, ChatHub, Middleware, Program.cs — depends on Application

Dependency Flow



▶ **Dependency Inversion** The Application layer owns IMessageRepository, ICacheService, and IMessageQueueService as interfaces. Infrastructure implements them. Application services never reference EF Core, Redis, or RabbitMQ — only the abstractions they define.

Runtime Component Map


JWT Validation
Controllers
ChatHub
ExceptionMiddleware
(outermost wrapper)
Application Services
ChatService
GroupService
PresenceService
SQL Server
(via EF)
Repositories
(EF Core)
RedisCacheService
(Presence TTL)
RabbitMQ
IMessageQueueService
ChatService

3. API Layer

Middleware Pipeline

ASP.NET Core processes every request through an ordered middleware pipeline. Each middleware wraps everything below it — order is non-negotiable:

```
  Request
     |
     ▼  ① ExceptionHandlingMiddleware   ← outermost; wraps all downstream
exceptions
     ▼  ② UseHttpsRedirection           ← security baseline before any processing
     ▼  ③ UseSwagger                    ← development only; before auth
     ▼  ④ UseCors                       ← before auth; OPTIONS preflights need no
JWT
     ▼  ⑤ UseAuthentication             ← reads JWT; populates HttpContext.User
     ▼  ⑥ UseAuthorization              ← checks [Authorize] attributes
     ▼  ⑦ MapControllers / MapHub       ← endpoint execution
     |
  Response
```

Global Exception Handling

A single middleware wraps the entire pipeline and converts all unhandled exceptions into structured JSON responses. Action filters are insufficient — they only cover controller execution; exceptions from SignalR hubs or other middleware would bypass them.

Exception Type	HTTP Status	Behaviour
DomainException	400	Message surfaced to caller — business-rule violations only
UnauthorizedAccessException	401	Generic message — no internal detail exposed
KeyNotFoundException	404	Entity lookup failure from Application layer
OperationCanceledException	499	Client disconnected — not a server error
All others	500	Generic message; full detail logged server-side only

Production Rule Stack traces, EF Core SQL text, and schema details are never returned to callers. They are logged via Serilog and visible only to operations teams. DomainException is the only type whose message crosses the API boundary.

Controller Design: Thin by Principle

Every controller method follows a strict four-step pattern: (1) extract caller identity from JWT claims, (2) stamp identity fields onto the DTO, (3) delegate to an Application service, (4) return the HTTP response. No exceptions.

Stamping SenderId from the JWT claim — never from the request body — is the single pattern that prevents message impersonation across the entire system:

```
// Client supplies { conversationId, body } only.
```

```
  // SenderId is always taken from the verified JWT claim.
  var callerId = GetCallerUserId();
  var request  = dto with { SenderId = callerId };  // body value silently
discarded
```

API Versioning & Swagger

URL segment versioning is used: /api/v{version}/[controller]. The version is explicit in logs, browser history, and every HTTP request — no custom headers or content negotiation required.

Swagger UI is enabled in Development only. OpenAPI documentation exposes endpoint structure, parameter shapes, and authentication schemes — serving it publicly in production is a reconnaissance vector. Internal teams access it through the development environment.

4. SignalR Architecture

Connection Lifecycle

```
  Client connects → wss://host/hubs/chat?access_token={jwt}
         │
         ▼  OnConnectedAsync()
         ├─ PresenceService.MarkOnlineAsync()          → SET user:presence:{id} EX
45
         └─ ChatService.DeliverPendingMessagesAsync()   → bulk Sent → Delivered
sweep
         │
  Client calls JoinConversation(conversationId)
         └─ Groups.AddToGroupAsync("conversation:{id}")
         │
  Client calls SendMessage(dto)
         ├─ ChatService.SendMessageAsync()             → persist then publish
         └─ Clients.Group().SendAsync("ReceiveMessage") → broadcast to group
         │
  Client disconnects (clean or network drop)
         ▼  OnDisconnectedAsync()
         ├─ PresenceService.MarkOfflineAsync()         → DEL key + record LastSeen
         └─ Redis TTL handles sudden drops independently (45s expiry)
```

Persist-Before-Broadcast

The database write always precedes the SignalR broadcast. This ordering prevents ghost messages — messages visible in a client's UI that do not exist in the database due to a failed write.

```
  ChatHub.SendMessage(dto)
         │
         ▼  STEP 1 — Persist (source of truth first)
  ChatService.SendMessageAsync()
    ├─ Message.Create()              ← Domain validates and constructs
    ├─ IMessageRepository.AddAsync() ← SQL Server write
    └─ IMessageQueueService.Publish() ← RabbitMQ event after successful persist
         │
         ▼  STEP 2 — Broadcast (only on success)
  Clients.Group("conversation:{id}").SendAsync("ReceiveMessage", dto)
    └─ Redis backplane fans out to ALL connected server instances
```

Redis SignalR Backplane

When the API runs on multiple server instances, SignalR group memberships exist only in the in-process memory of each instance. Without coordination, a broadcast on Server A cannot reach clients connected to Server B.

The Redis backplane uses pub/sub to synchronise group broadcasts across all instances. Each instance subscribes to the same channel namespace. Adding a server instance requires no configuration change — it connects to the same Redis prefix and receives all group events automatically.

```
  User A (Server 1) ──┐
  User B (Server 1) ──▶  Group: conversation:abc  →  Redis pub/sub channel
  User C (Server 2) ──▶  All instances receive;  relay to local connections
  User D (Server 3) ──┘
```

Typing Indicators & Heartbeat

Typing signals are ephemeral transport events — never persisted. The hub receives NotifyTyping(conversationId) and immediately relays it to Clients.OthersInGroup(group). No service call, no database touch, no queue event.

The Redis presence key (user:presence:{id}) has a 45-second TTL. Clients send a Heartbeat hub call every 30 seconds, which calls PresenceService.MarkOnlineAsync() to reset the TTL. If no heartbeat arrives within 45 seconds — network drop, battery death, process crash — the key expires automatically. The user is effectively offline without any cleanup job or explicit offline notification.

5. Authentication & Authorization

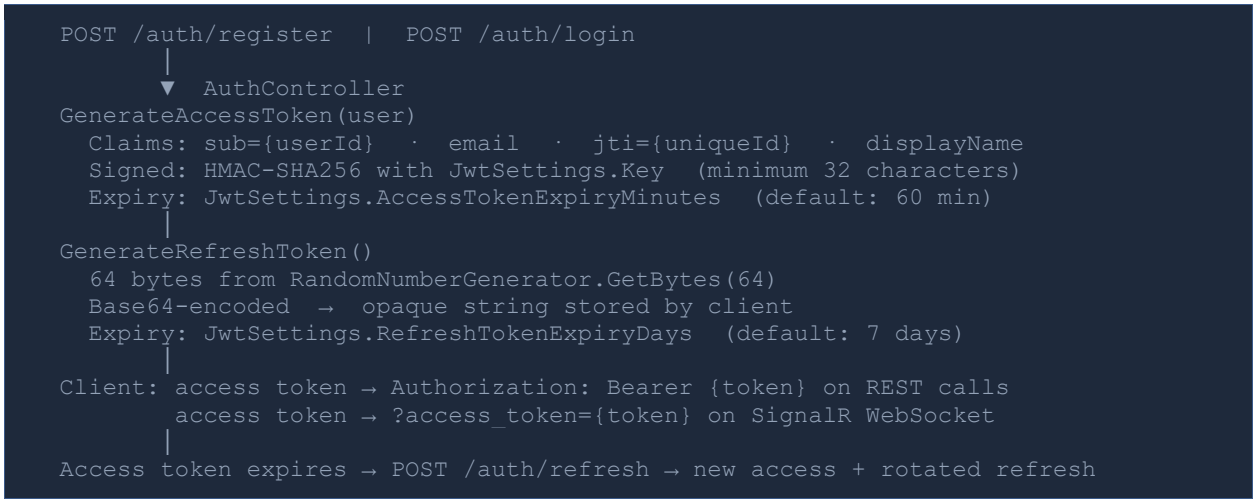
JWT Validation

All four standard JWT validations are enforced. None are optional in production:

Validation	Risk if Disabled	Configuration
Issuer	Staging tokens accepted in production	ValidIssuer from JwtSettings
Audience	Cross-API privilege escalation	ValidAudience from JwtSettings
Lifetime	Expired tokens work indefinitely	ValidateLifetime = true
Signature	Any token accepted — completely catastrophic	HMAC-SHA256 with 256-bit key

Clock Skew: ClockSkew = TimeSpan.Zero is explicitly set. The ASP.NET Core default is 5 minutes, quietly extending the validity window of any stolen token beyond its stated expiry. Zero skew forces proper refresh token rotation.

Token Lifecycle



Password Security & Identity Stamping

BCrypt with work factor 12 is used for password hashing. The raw password never enters the Application or Domain layers — it is hashed in AuthController before the User entity is constructed. The domain entity stores only the hash string.

Login returns the same error message for "email not found" and "wrong password" to prevent user enumeration. An attacker cannot determine which email addresses are registered from login response differences.

SignalR WebSocket and SSE connections cannot send custom HTTP headers. The access token is transmitted as a query parameter (?access_token=...) and extracted by an OnMessageReceived event hook, which moves it to the Authorization header before the standard JWT middleware validates it. No changes to the validation pipeline are required.

6. Application Layer

The Application layer contains all use-case orchestration logic. It coordinates domain objects, repositories, cache, and messaging without containing any business rules itself. Business rules live in the Domain; infrastructure concerns live in Infrastructure. The Application layer is the composition boundary between them.

Service Responsibilities

Service	Responsibility
ChatService	Message send/receive lifecycle: validate → guard → create (Domain) → persist → publish → return DTO
GroupService	Conversation lifecycle: create direct/group, add/remove participants, rename, return inbox list
PresenceService	Online/offline state via Redis TTL keys: mark online, mark offline, batch online-status query

SendMessageAsync — Orchestration Flow

```
SendMessageAsync(SendMessageDto dto)
    │
    ├─ 1.  SendMessageValidator.Validate(dto)           → fail fast, no I/O
    ├─ 2.  IUserRepository.GetByIdAsync(dto.SenderId)   → sender existence guard
    ├─ 3.  IConversationRepo.GetByIdAsync(convId)       → conversation existence
    ├─ 4.  IsParticipantAsync(convId, senderId)         → membership authorization
    ├─ 5.  Message.Create(convId, senderId, body)       ← Domain constructs entity
    ├─ 6.  IMessageRepository.AddAsync(message)         → SQL Server persist
    ├─ 7.  conversation.UpdateLastMessageAt(sentAt)     ← Domain updates sort key
    ├─ 8.  IConversationRepo.UpdateAsync(conversation)  → persist updated
timestamp
    ├─ 9.  IMessageQueueService.PublishMessageAsync()   → RabbitMQ event publish
    └─ 10. MapToResponseDto(message, senderName)        → return DTO, never entity
```

ℹ **Layer Boundary Rule** Domain entities never cross the Application layer boundary. All methods return DTOs. This decouples the API contract from the internal domain model — changes to entity structure do not force API contract changes.

Dependency Inversion in Practice

Application services depend exclusively on interfaces they define: IMessageRepository, IUserRepository, IConversationRepository, ICacheService, IMessageQueueService. These interfaces are implemented in Infrastructure. At runtime the DI container resolves concrete implementations. In tests, mock implementations are injected with no database or external service required.

7. Domain Layer

The Domain layer contains all business entities, invariants, and state transition logic. It has zero external dependencies — no NuGet packages, no framework references, no EF Core attributes. It compiles and unit-tests in complete isolation. Every entity is testable with a single constructor call.

Entities & Key Design Decisions

Entity	Key Design Decisions
User	Private setters on all properties; mutation only via named methods; BCrypt hash stored, never raw password; soft-delete via IsActive flag preserves message history
Conversation	Factory methods CreateDirect() (Title=null enforced) and CreateGroup() (Title required); monotonic guard on LastMessageAt prevents backward movement under retry
Message	Status lifecycle Sent→Delivered→Read is one-directional and enforced; MarkAsDelivered/MarkAsRead are idempotent — safe under concurrent delivery; Delete() scrubs Body for data minimization
ConversationParticipant	LastReadAt drives unread count queries without a separate receipts table; MarkAsRead() is monotonic; admin promotion/demotion guards against no-op calls

Rich Domain Model vs Anemic Model

Every state change goes through a named method that validates before mutating. Entities are not passive data containers — they own their behavior:

```
// Anemic model — business rule lives outside the entity:
message.Status = MessageStatus.Delivered;   // no guard, no DeliveredAt
message.DeliveredAt = DateTime.UtcNow;

// Rich model — business rule enforced inside the entity:
message.MarkAsDelivered();
// Internally: validates not deleted, validates not already Read,
//             sets Status + DeliveredAt atomically, idempotent on repeat call.
```

Persistence Ignorance & DomainException

Domain entities have no navigation properties. No entity holds a List<Message> or a reference to DbContext. EF Core accesses private fields via reflection — the domain is unaware of this. The private parameterless constructor required by EF Core is marked private so application code cannot create an empty, invalid entity.

DomainException is the single error channel for business rule violations. It propagates unmodified from entity methods through the Application layer to ExceptionHandlingMiddleware, which maps it to HTTP 400. Infrastructure never throws DomainException.

8. Infrastructure Layer

EF Core Configuration

All entity-to-table mappings use Fluent API exclusively — zero Data Annotations on domain entities. Configurations are auto-discovered via ApplyConfigurationsFromAssembly(). Adding a new entity configuration class requires no changes to AppDbContext.

Decision	Rationale
AsNoTracking() on reads	Eliminates change-tracker snapshot overhead on projection queries where no mutation follows
Tracking on mutation reads	GetByIdAsync for update scenarios keeps tracking ON — entity is mutated then passed to UpdateAsync
ExecuteDeleteAsync	Issues a single DELETE without loading the entity first, avoiding a redundant SELECT round-trip
Global query filter	HasQueryFilter(!IsDeleted) on Messages auto-appends to every query — no caller can forget the filter
ValueGeneratedNever()	Application-side sequential GUIDs prevent clustered index fragmentation from random inserts
Enum as byte conversion	HasConversion<byte>() stores MessageStatus and ConversationType as TINYINT — compact, index-friendly

Service Lifetimes

Incorrect service lifetimes are one of the most common production bugs in ASP.NET Core applications. The registrations below are deliberate:

Service	Lifetime	Reason
AppDbContext	Scoped	One Unit of Work per request; shared across repositories in the same request
Repositories	Scoped	Depend on Scoped DbContext — registering as Singleton creates a captive dependency
IConnectionMultiplexer (Redis)	Singleton	Manages a connection pool; designed for application-lifetime sharing
ICacheService	Scoped	Stateless after construction; Scoped keeps it consistent with the repository tier
IConnection (RabbitMQ)	Singleton	Physical broker connections are expensive; channels are created from the singleton
IMessageQueueService	Singleton	Holds one channel; implements IDisposable for clean shutdown ordering

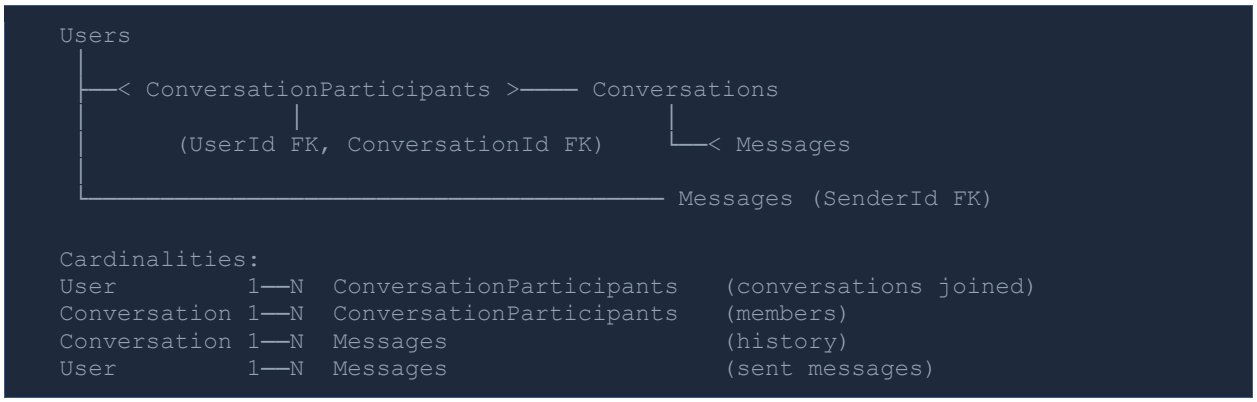
Captive Dependency Registering a Scoped service (e.g. DbContext) as Singleton causes the

Service	Lifetime	Reason

container to permanently capture one instance. For DbContext this means a single instance shared across all concurrent requests — threading exceptions and data corruption. The lifetimes above prevent this.

9. Database Design

Schema & Relationships



Key Column Design Decisions

Column	Design Decision
Conversations.LastMessageAt	Denormalized from MAX(SentAt). Updated on every message write. Eliminates GROUP BY aggregation on every inbox load — the highest-frequency read query in the system.
ConversationParticipants.LastReadAt	Drives unread count: COUNT(*) WHERE SentAt > LastReadAt. Eliminates a separate MessageReadReceipts table for single-device scenarios.
Messages.Body NVARCHAR(4000)	Fits within a single 8KB SQL Server data page. NVARCHAR(MAX) uses LOB allocation units requiring a second disk read per row. 4000 characters covers any realistic chat message.
Messages.IsDeleted + global filter	EF Core global query filter auto-appends WHERE IsDeleted = 0 to every query. Soft delete preserves timeline coherence; hard delete creates visible gaps.
All PKs: GUID, ValueGeneratedNever()	Application-generated sequential GUIDs. Avoids clustered index fragmentation from random NEWID() inserts. IDs exist before the database write — useful for optimistic UI patterns.

Indexing Strategy

Index	Query Served
IX_Messages_ConversationId_SentAt DESC (covering)	Chat history cursor pagination — no key lookup needed; entire query from index
IX_CP_UserId_ConversationId INCLUDE(LastReadAt)	Inbox load — what conversations is this user in?
IX_CP_ConversationId_UserId	Membership check on every message send (IsParticipantAsync)
IX_Conversations_LastMessageAt DESC	Inbox sort — ORDER BY LastMessageAt DESC without a sort step

Index	Query Served
UQ_Users_Username, UQ_Users_Email	DB-level uniqueness — final guard beyond application checks
UQ_CP_(ConversationId, UserId)	Prevents duplicate membership regardless of application-layer bugs

Cursor Pagination vs OFFSET

Message history pagination uses SentAt as a seek predicate, not page numbers:

```
-- OFFSET (DO NOT USE — linear cost growth):
SELECT * FROM Messages WHERE ConversationId = @id
ORDER BY SentAt DESC
OFFSET 5000 ROWS FETCH NEXT 50 ROWS ONLY
-- Scans and discards 5000 rows on every page-101 request.

-- Cursor seek (production correct — O(log N) at any depth):
SELECT TOP 50 * FROM Messages
WHERE ConversationId = @id AND SentAt < @cursorSentAt
ORDER BY SentAt DESC
-- Index seek directly to cursor position via covering index.
```

10. Redis & RabbitMQ

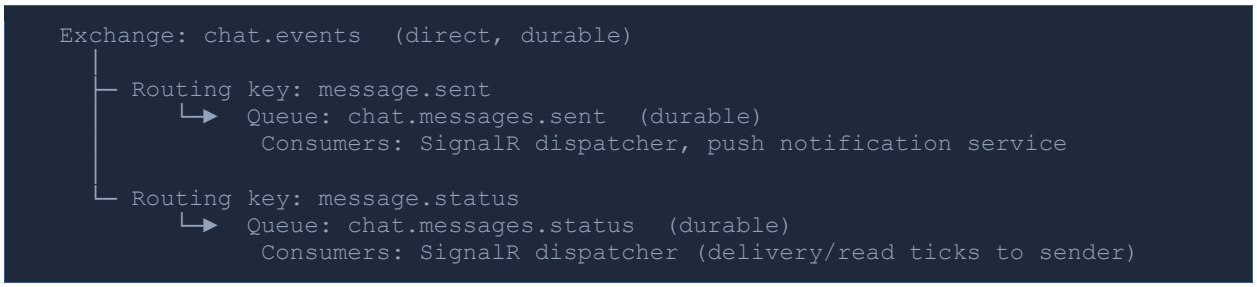
Redis: Presence Tracking

Key Pattern	Purpose & TTL
user:presence:{userId}	Online status — key existence = online; 45-second TTL; expiry = offline
user:lastseen:{userId}	"Last seen X ago" display — 30-day TTL; written on clean disconnect
ChatSystem.* (SignalR)	Backplane pub/sub channel; TTL managed by SignalR framework

The presence model deliberately avoids explicit "offline" writes for sudden disconnects. TCP keepalive can delay drop detection by 30+ seconds; relying on OnDisconnectedAsync alone would show users as online after network failures. The two-mechanism approach — immediate DEL on clean disconnect, 45-second TTL expiry on drops — ensures accurate presence through both paths.

At 10,000 concurrent users with 30-second heartbeat intervals, presence writes generate approximately 333 writes per second. Redis handles O(1) SET/DEL operations at this frequency with sub-millisecond latency. Writing this volume to SQL Server would add unnecessary IOPS to the primary database, increase index maintenance overhead, and compete with message history queries.

RabbitMQ: Async Event Pipeline



Three properties combine for at-least-once delivery: durable exchange + durable queues (topology survives broker restart), persistent delivery mode / DeliveryMode = 2 (messages written to disk before ACK), and a unique MessageId per publish (enables consumer-side idempotency checks under retry).

Calling push notification services and analytics consumers synchronously from ChatService would create temporal coupling — a slow consumer delays message delivery for all users. The queue decouples the write path from all consumers: each processes at its own pace, and a consumer being temporarily down does not affect message delivery or the sender's experience.

11. Security Considerations

Secret Management & Configuration

No secrets belong in appsettings.json committed to source control. The following are supplied via environment variables or a secrets manager (Azure Key Vault, AWS Secrets Manager) in production:

Environment Variable	Description
Jwt__Key	HMAC-SHA256 signing key — minimum 32 characters (256 bits); validated at startup
ConnectionStrings__SqlServer	Full connection string including credentials
ConnectionStrings__Redis	Redis connection string
RabbitMq__Password	RabbitMQ broker password

ASP.NET Core's configuration system merges environment variables over appsettings.json automatically. JwtSettings key length is validated at startup — the application refuses to start with a missing or undersized key.

Security Boundaries

- All hub and controller endpoints require JWT via [Authorize] — only /auth/* and /health are unauthenticated
- SenderId is always stamped from JWT claims — client-supplied sender identity is silently overwritten
- HTTPS enforced via UseHttpsRedirection() — HTTP requests receive a 301 redirect before any processing
- CORS in production locks to specific allowed origins; AllowCredentials() cannot be combined with AllowAnyOrigin() — a deliberate framework constraint
- Swagger UI disabled outside Development — OpenAPI documentation reveals endpoint structure to attackers
- 500 responses return a generic message only — internal details logged server-side exclusively
- Login returns identical error messages for "not found" and "wrong password" — prevents user enumeration

12. Production Architecture

Horizontal Scaling Model

The API layer is stateless by design. Adding an instance requires pointing it at the same SQL Server, Redis, and RabbitMQ. The load balancer needs no sticky session configuration.

Component	Stateless Mechanism	Shared State Location
HTTP Auth	JWT carries identity — no server-side session	None required
SignalR groups	Redis backplane synchronises across all instances	Redis pub/sub
Presence	Redis TTL keys readable from any instance	Redis
Message data	SQL Server — single source of truth	SQL Server
Event processing	RabbitMQ — consumers can run on any instance	RabbitMQ queues

Docker Compose Infrastructure

```
services:
  api:        ASP.NET Core 8  →  horizontal replicas possible
  sqlserver:  SQL Server 2022 →  primary data store with volume persistence
  redis:      Redis 7-alpine  →  presence tracking + SignalR backplane
  rabbitmq:   RabbitMQ 3      →  event queue with management UI on port 15672

Health checks on all infrastructure services:
  sqlserver:  sqlcmd -Q "SELECT 1"
  redis:      redis-cli ping
  rabbitmq:   rabbitmq-diagnostics ping

api depends_on with condition: service_healthy
→ API does not start until all infrastructure is confirmed ready.
```

Failure Scenarios

Failure	System Behaviour
API instance restarts	Clients reconnect; OnConnectedAsync delivers pending messages; no data loss
Redis unavailable	Presence and SignalR backplane degrade; messages still persist to SQL Server
RabbitMQ unavailable	Queue publish fails after DB write; retry worker re-publishes when broker recovers
SQL Server unreachable	EF Core retry policy (5 attempts, 10s max delay); DomainException surfaces after exhaustion
Network drop (client)	Redis TTL expires after 45s; user shown as offline; reconnect triggers delivery sweep

13. Request & Event Flows

User Login

```
POST /api/v1/auth/login  { email, password }
  │
  ▼  AuthController.Login()
  ├─ IUserRepository.GetByEmailAsync(email)
  ├─ BCrypt.Verify(password, user.PasswordHash)
  ├─ GenerateAccessToken(user)    → HMAC-SHA256 JWT, 60-min expiry
  ├─ GenerateRefreshToken()       → 64 cryptographically random bytes
  └─ Return 200  { accessToken, refreshToken, expiresAt, userId, displayName }

Note: identical error response for "not found" and "wrong password".
```

Sending a Message (REST & SignalR)

```
REST:     POST /api/v1/messages  →  MessagesController.SendMessage()
SignalR:  ChatHub.SendMessage(dto)

Both paths converge at ChatService.SendMessageAsync():
  │
  ├─ 1.  SendMessageValidator.Validate()          → fail fast
  ├─ 2.  IUserRepository.GetByIdAsync()           → sender guard
  ├─ 3.  IConversationRepo.GetByIdAsync()         → conversation guard
  ├─ 4.  IsParticipantAsync()                     → membership auth
  ├─ 5.  Message.Create()                         ← Domain constructs
  ├─ 6.  IMessageRepository.AddAsync()            → SQL Server write
  ├─ 7.  conversation.UpdateLastMessageAt()       ← Domain updates sort key
  ├─ 8.  IConversationRepo.UpdateAsync()          → persist timestamp
  ├─ 9.  IMessageQueueService.PublishAsync()      → RabbitMQ event
  └─ 10. MapToResponseDto()                       → return DTO

SignalR only (after Step 10):
  Clients.Group("conversation:{id}").SendAsync("ReceiveMessage", dto)
  └─ Redis backplane fans out to all server instances
```

Offline Message Delivery on Reconnect

```
User reconnects  →  ChatHub.OnConnectedAsync()
  │
  ▼  PresenceService.MarkOnlineAsync()          → refresh Redis TTL
  ▼  ChatService.DeliverPendingMessagesAsync(userId)
  │
  ├─ IMessageRepo.GetUndeliveredMessagesForUserAsync(userId)
  │    → Messages where Status = Sent
  │    → SenderId ≠ userId
  │    → ConversationId in user's conversations
  │
  └─ For each pending message:
       message.MarkAsDelivered()                ← Domain: Sent → Delivered
       IMessageRepository.UpdateAsync()         → persist Status + DeliveredAt
       IMessageQueueService.PublishStatusUpdate() → notify original sender
```

14. Engineering Decisions

This section documents the reasoning behind the most significant architectural choices. Each decision was made to address a specific engineering constraint.

Clean Architecture

The primary benefit is testability and replaceability. The Domain layer has zero dependencies — every entity is unit-testable without a database or web server. The Application layer's interfaces allow any infrastructure component to be replaced without modifying any layer above it. SQL Server can be swapped for PostgreSQL, Redis for Memcached, or RabbitMQ for Azure Service Bus by replacing Infrastructure implementations alone. In a long-lived system this is not academic — it is the difference between a codebase that can evolve and one that cannot.

SignalR over Raw WebSockets

Raw WebSockets provide a transport pipe. Group management, multi-transport fallback (WebSocket → SSE → Long Polling), reconnection handling, and multi-server coordination built on raw WebSockets would require weeks of infrastructure work. SignalR provides these as first-class primitives that map directly to chat system concepts. The Redis backplane for horizontal scaling is a single method registration at startup.

Redis for Presence

Presence is high-frequency ephemeral state with no long-term business value. At 10,000 concurrent users with 30-second heartbeat intervals, presence writes approach 333 operations per second. SQL Server is an ACID-compliant relational engine optimised for durable, relational queries — not for high-frequency expiring boolean flags. Redis TTL keys model the lifecycle naturally: key existence = online, TTL expiry = offline. The expiry mechanism replaces an entire cleanup background service.

RabbitMQ for Event Processing

Calling push notification services and analytics consumers synchronously from ChatService creates temporal coupling — a slow or unavailable downstream service delays message delivery for all users. Publishing to a queue decouples the write path from all consumers. Each consumer processes at its own pace; a consumer being temporarily down does not affect message delivery. The durability guarantees (durable queue + persistent delivery mode) ensure messages survive a broker restart between publish and consumer acknowledgement.

Cursor Pagination

OFFSET-based pagination requires scanning and discarding all prior rows on every request. For a conversation with 50,000 messages, loading page 1,000 discards 49,950 rows — cost grows linearly with history depth. Cursor pagination using a seek predicate on the indexed SentAt column is O(log N) at any depth. For a system where users routinely scroll through long message histories, this is the correct baseline design, not an optimisation.

Middleware Exception Handling

Action filters wrap controller execution only — exceptions from SignalR hubs, other middleware, or startup code bypass them entirely. Placing exception handling at the outermost middleware position wraps the complete pipeline. There is no execution path that bypasses it. One exception-

to-status mapping, one response shape, enforced unconditionally across REST and real-time paths.

Soft Delete on Messages

Hard-deleting a message creates a visible gap in the conversation timeline. References to the deleted message in client-side caches produce inconsistency, and the message content may be required for abuse investigation or legal hold workflows. Soft delete (IsDeleted = true, Body scrubbed to empty string) preserves timeline coherence while removing the content. The EF Core global query filter ensures deleted messages are automatically excluded from all queries without any caller needing to remember the filter condition.

15. Appendix

Project Folder Structure

```
RealtimeChatSystem/
├── src/
│   ├── ChatSystem.Domain/
│   │   ├── Entities/          User · Conversation · Message · ConversationParticipant
│   │   ├── Enums/             MessageStatus · ConversationType
│   │   └── Exceptions/        DomainException
│   │
│   ├── ChatSystem.Application/
│   │   ├── DTOs/              ChatDtos.cs  (all inbound + outbound shapes)
│   │   ├── Interfaces/        IMessageRepo · IUserRepo · IConvRepo · ICacheService · IQueueService
│   │   ├── Services/          ChatService · GroupService · PresenceService
│   │   └── Validators/        SendMessageValidator · CreateGroupConversationValidator
│   │
│   ├── ChatSystem.Infrastructure/
│   │   ├── Caching/           RedisCacheService
│   │   ├── DependencyInjection/  InfrastructureServiceExtensions
│   │   ├── Messaging/         MessageEvents · RabbitMqMessageQueueService
│   │   └── Persistence/
│   │       ├── AppDbContext.cs
│   │       ├── Configurations/  UserConfig · ConversationConfig · MessageConfig · CPConfig
│   │       └── Repositories/    MessageRepository · UserRepository · ConversationRepository
│   │
│   └── ChatSystem.API/
│       ├── Controllers/       AuthController · MessagesController · ConversationsController
│       ├── Extensions/        ApplicationServiceExtensions · AuthServiceExtensions · SwaggerServiceExtensions
│       ├── Hubs/              ChatHub
│       ├── Middleware/        ExceptionHandlingMiddleware
│       ├── Settings/          JwtSettings
│       └── Program.cs
│
├── docker-compose.yml
└── README.md
```

REST API Reference

Method	Endpoint	Status	Description
POST	/api/v1/auth/register	201	Register — returns JWT pair
POST	/api/v1/auth/login	200	Authenticate — returns JWT pair
POST	/api/v1/auth/refresh	200	Exchange refresh token
GET	/api/v1/conversations	200	Inbox — sorted by LastMessageAt
POST	/api/v1/conversations/direct	201	Start or retrieve 1-to-1 conversation
POST	/api/v1/conversations/group	201	Create group

Method	Endpoint	Status	Description
			conversation
POST	/api/v1/conversations/participants	204	Add participant (admin only)
DELETE	/api/v1/conversations/{id}/participants/{uid}	204	Remove participant or leave group
PATCH	/api/v1/conversations/{id}/rename	204	Rename group (admin only)
POST	/api/v1/messages	201	Send message via REST
GET	/api/v1/messages/conversations/{id}	200	Paginated message history
DELETE	/api/v1/messages/{id}	204	Soft-delete (sender only)
PATCH	/api/v1/messages/conversations/{id}/read	204	Mark conversation as read
GET	/health	200	Liveness probe

SignalR Hub Methods

Method	Direction	Description
SendMessage	Client → Server	Persist and broadcast a message to the conversation group
JoinConversation	Client → Server	Add connection to SignalR group; notify group of online presence
LeaveConversation	Client → Server	Remove connection from group; notify group
NotifyTyping	Client → Server	Fan-out typing signal to OthersInGroup — not persisted
NotifyStoppedTyping	Client → Server	Fan-out stop-typing signal — not persisted
Heartbeat	Client → Server	Refresh Redis presence TTL; server responds with Pong
ReceiveMessage	Server → Client	Delivers MessageResponseDto to all group members
MessageStatusUpdated	Server → Client	Notifies sender of Delivered or Read status change
UserOnline/Offline	Server → Client	Presence change notification within a conversation group
UserTyping/Stopped	Server → Client	Typing indicator relay to other group members

Key Environment Variables

```
# Required in all non-development environments:
Jwt__Key                        HMAC-SHA256 key, minimum 32 characters
Jwt__Issuer                     Token issuer claim value
Jwt__Audience                   Token audience claim value
```

```
  ConnectionStrings__SqlServer    Full SQL Server connection string
  ConnectionStrings__Redis        Redis connection string
  RabbitMq__Host                  RabbitMQ broker hostname
  RabbitMq__Password              RabbitMQ broker password

  # Startup validation:
  Jwt__Key is validated at startup — app refuses to start if missing or < 32
chars.
  ConnectionStrings__Redis is validated before SignalR backplane registration.
```